

146. LRU Cache Medium

Acceptance: 46.6% | [LeetCode](#)

Design a data structure that follows the constraints of a [Least Recently Used \(LRU\) cache](#).

Implement the `LRUCache` class:

- `LRUCache(int capacity)` Initialize the LRU cache with **positive** size `capacity`.
- `int get(int key)` Return the value of the `key` if the key exists, otherwise return `-1`.
- `void put(int key, int value)` Update the value of the `key` if the `key` exists. Otherwise, add the `key-value` pair to the cache. If the number of keys exceeds the `capacity` from this operation, **evict** the least recently used key.

The functions `get` and `put` must each run in `O(1)` average time complexity.

Example 1:

```
**Input**
["LRUCache", "put", "put", "get", "put", "get", "put",
 "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1],
```

```
[3], [4]]
**Output**
[null, null, null, 1, null, -1, null, -1, 3, 4]

**Explanation**
LRUCache lruCache = new LRUCache(2);
lruCache.put(1, 1); // cache is {1=1}
lruCache.put(2, 2); // cache is {1=1, 2=2}
lruCache.get(1);    // return 1
lruCache.put(3, 3); // LRU key was 2, evicts key 2,
                    // cache is {1=1, 3=3}
lruCache.get(2);    // returns -1 (not found)
lruCache.put(4, 4); // LRU key was 1, evicts key 1,
                    // cache is {4=4, 3=3}
lruCache.get(1);    // return -1 (not found)
lruCache.get(3);    // return 3
lruCache.get(4);    // return 4
```

Constraints:

- `1 <= capacity <= 3000`
- `0 <= key <= 104`
- `0 <= value <= 105`
- At most `2 * 105` calls will be made to `get` and `put`.

Claude Opus Solution

LRU Cache — Complete Solution

1. Intuition

The challenge is achieving **O(1)** for *both* operations. This requires two needs:

1. **Fast lookup by key** → suggests a **hash map** (`dict`).
2. **Fast ordering updates** (move item to "most recently used", evict "least recently used") → suggests a **doubly linked list**.

Key insight: Neither structure alone works. - A hash map gives `O(1)` lookup but has no concept of "order of use." - A linked list maintains order, but finding an arbitrary node is `O(n)`.

Combine them: Store nodes in a doubly linked list ordered by recency. Use the hash map to map `key → node`, so you can jump to any node in `O(1)`. The doubly linked list lets you unlink/relink a node in `O(1)` because each node knows its `prev` and `next`.

```
MRU (most recent) <-----> ... <-----> LRU (least recent)
    head_dummy                      tail_dummy
```

- **Head side** = most recently used.
- **Tail side** = least recently used (first to be evicted).

2. Approach (Pseudocode)

```
INIT(capacity):
    dict = {}
    create dummy head & dummy tail, link them together    # avoids edge-case null checks

GET(key):
    if key not in dict: return -1
    node = dict[key]
    remove node from its position
    insert node right after head        # mark as most recently used
    return node.value
```

```

PUT(key, value):
    if key in dict:
        remove old node (will re-add fresh)
    create new node, store in dict
    insert node right after head      # most recently used
    if size > capacity:
        lru = tail.prev              # node before dummy tail
        remove lru from list
        delete dict[lru.key]

```

Helper operations (the heart of $O(1)$): - `remove(node)` : connect `node.prev` directly to `node.next` . - `insert_front(node)` : splice node between head and head's current next.

3. Python Solution

```

class ListNode:
    """Node of a doubly linked list. Stores both key and value.
    We store the key so that when we evict the tail (LRU) node,
    we can also delete it from the hash map in  $O(1)$ ."""
    def __init__(self, key=0, value=0):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = {}          # key -> ListNode, for  $O(1)$  lookup

        # Dummy head and tail sentinels.
        # head <-> ... <-> tail. This removes the need to handle
        # empty-list / first-node / last-node edge cases.
        self.head = ListNode()   # head.next = most recently used
        self.tail = ListNode()   # tail.prev = least recently used
        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node: ListNode) -> None:
        """Unlink a node from the list in  $O(1)$ ."""
        prev_node, next_node = node.prev, node.next
        prev_node.next = next_node
        next_node.prev = prev_node

    def _insert_front(self, node: ListNode) -> None:
        """Insert node right after head => mark as most recently used."""
        first = self.head.next   # current MRU node
        # Wire node between head and first
        node.prev = self.head
        node.next = first
        self.head.next = node
        first.prev = node

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        node = self.cache[key]
        # Accessing => becomes most recently used: move to front.
        self._remove(node)
        self._insert_front(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        # If key exists, remove the old node so we can re-insert fresh at front.
        if key in self.cache:
            self._remove(self.cache[key])

        # Create and register the new node, then place at front (MRU).
        node = ListNode(key, value)
        self.cache[key] = node

```

```

self._insert_front(node)

# If we exceeded capacity, evict the LRU node (just before tail).
if len(self.cache) > self.capacity:
    lru = self.tail.prev      # least recently used node
    self._remove(lru)        # unlink from list
    del self.cache[lru.key]   # remove from hash map (key stored on node!)

```

4. Complexity Analysis

Operation	Time	Why
get	O(1)	dict lookup + constant linked-list pointer surgery
put	O(1)	dict lookup/insert/delete + constant pointer surgery

- **Space Complexity: O(capacity)** — We store at most `capacity` entries in both the hash map and the doubly linked list.

Why O(1) and not O(n)? The hash map gives us *direct access* to any node, so we never traverse the list to find a node. All list updates (`_remove`, `_insert_front`) only touch a fixed number of neighboring pointers.

5. Alternative Approaches

A. `collections.OrderedDict` (Pythonic shortcut)

`OrderedDict` is itself backed by a hash map + doubly linked list, so it gives us the ordering and O(1) operations "for free."

```

from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = OrderedDict()

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)      # mark as most recently used
        return self.cache[key]

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False) # pop the LRU (front) item

```

- **Pros:** Very concise. **Cons:** Hides the underlying mechanics; interviewers often want the manual version to test data-structure knowledge.

B. Hash map + Python `list` (NOT recommended)

Use a dict for values and a list to track order. **Problem:** removing/moving an element in a list is **O(n)**, violating the O(1) requirement. Good to know as a "naive" baseline to contrast with.

C. Comparison with the reference solution

The reference solution cleverly reuses `self.capacity` as a *running counter* (decrementing on insert, incrementing on remove) to detect when eviction is needed. My version instead checks `len(self.cache) > capacity`, which is arguably clearer and keeps `capacity` immutable. Both are valid O(1) approaches with identical asymptotic behavior.

Codex Solution

1. Intuition

An LRU cache needs to support:

1. `get(key)` in O(1)
2. `put(key, value)` in O(1)

3. Quickly find and remove the **least recently used** item

The key observation is:

- A **hash map** gives $O(1)$ access to a key.
- A **doubly linked list** lets us move nodes and remove nodes in $O(1)$.

So we combine them:

- Hash map: `key -> node`
- Doubly linked list: stores nodes in usage order

We keep:

- Most recently used node near the **head**
- Least recently used node near the **tail**

Whenever we access or update a key, we move that node to the front.

When capacity is exceeded, we remove the node at the back.

2. Approach

Data structures

Each node stores:

```
key
value
prev pointer
next pointer
```

The cache stores:

```
capacity
hash map: key -> node
dummy head
dummy tail
```

We use dummy head and dummy tail to avoid edge cases when inserting/removing nodes.

```
head <=> most recent <=> ... <=> least recent <=> tail
```

Helper functions

Remove a node from the linked list

```
remove(node):
    node.prev.next = node.next
    node.next.prev = node.prev
```

Insert a node right after head

```
insert_after_head(node):
    node.next = head.next
    node.prev = head

    head.next.prev = node
    head.next = node
```

Move a node to the front

```
move_to_front(node):
    remove(node)
    insert_after_head(node)
```

get(key)

```
get(key):
    if key does not exist in map:
        return -1

    node = map[key]
    move node to front because it was recently used
    return node.value
```

put(key, value)

```
put(key, value):
    if key already exists:
        node = map[key]
        update node.value
        move node to front
        return

    create new node
    add it to map
    insert it after head

    if cache size exceeds capacity:
        lru = node before tail
        remove lru from list
        delete lru.key from map
```

3. Python Solution

```
class Node:
    """
    A node in the doubly linked list.

    Each node stores:
    - key: needed so we can remove it from the hashmap during eviction
    - value: the cached value
    - prev/next: pointers for O(1) removal and insertion
    """

    def __init__(self, key=0, value=0):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None


class LRUCache:
    def __init__(self, capacity: int):
        """
        Initialize the LRU cache.

        We use:
        - self.cache: maps key -> corresponding linked list node
        - self.head and self.tail: dummy nodes to simplify insertion/removal

        The linked list order is:

        head <=> most recently used <=> ... <=> least recently used <=> tail
        """

        self.capacity = capacity
        self.cache = {}

        # Dummy head and tail nodes.
        # These do not store real cache entries.
        self.head = Node()
        self.tail = Node()

        # Initially, the list is empty:
```

```

# head <=> tail
self.head.next = self.tail
self.tail.prev = self.head

def _remove(self, node: Node) -> None:
    """
    Remove an existing node from the doubly linked list in O(1).

    Before:
        prev <=> node <=> nxt

    After:
        prev <=> nxt

    """
    prev_node = node.prev
    next_node = node.next

    prev_node.next = next_node
    next_node.prev = prev_node

def _insert_after_head(self, node: Node) -> None:
    """
    Insert a node right after the dummy head in O(1).

    Since the node is placed at the front, it becomes the most recently used.
    """
    current_first = self.head.next

    node.prev = self.head
    node.next = current_first

    self.head.next = node
    current_first.prev = node

def _move_to_front(self, node: Node) -> None:
    """
    Move an existing node to the front.

    This marks it as the most recently used.
    """
    self._remove(node)
    self._insert_after_head(node)

def get(self, key: int) -> int:
    """
    Return the value associated with key if it exists.

    If key exists:
    - Move it to the front because it was just used.
    - Return its value.

    If key does not exist:
    - Return -1.
    """
    if key not in self.cache:
        return -1

    node = self.cache[key]

    # Accessing the node makes it recently used.
    self._move_to_front(node)

    return node.value

def put(self, key: int, value: int) -> None:
    """
    Insert or update a key-value pair.

    Cases:

```

1. Key already exists:
 - Update the value.
 - Move node to front.
 2. Key does not exist:
 - Create a new node.
 - Insert it at the front.
 - Add it to the hashmap.
 - If capacity is exceeded, remove the least recently used node.
-

```
if key in self.cache:
    # Update existing node.
    node = self.cache[key]
    node.value = value

    # Since this key was just used, move it to the front.
    self._move_to_front(node)
    return

# Create a new node for this key-value pair.
new_node = Node(key, value)

# Add it to both the linked list and hashmap.
self.cache[key] = new_node
self._insert_after_head(new_node)

# If we exceeded capacity, evict the least recently used node.
if len(self.cache) > self.capacity:
    # The least recently used node is right before dummy tail.
    lru_node = self.tail.prev

    # Remove it from the linked list.
    self._remove(lru_node)

    # Remove it from the hashmap.
    del self.cache[lru_node.key]
```

4. Complexity Analysis

Let n be the capacity of the cache.

get(key)

Hash map lookup:

$O(1)$

Moving a node in a doubly linked list:

$O(1)$

Therefore:

Time Complexity: $O(1)$

put(key, value)

Hash map lookup/insertion/deletion:

$O(1)$

Linked list insertion/removal:

$O(1)$

Therefore:

Time Complexity: $O(1)$

Space Complexity

We store at most `capacity` nodes.

Each node is also stored in the hash map.

Space Complexity: $O(\text{capacity})$

5. Alternative Approaches

1. Python `collections.OrderedDict`

Python has a built-in ordered dictionary that can solve this problem more simply.

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = OrderedDict()

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1

        # Move key to the end to mark it as recently used.
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            # Update existing key and mark as recently used.
            self.cache.move_to_end(key)

        self.cache[key] = value

        if len(self.cache) > self.capacity:
            # popitem(last=False) removes the least recently used item.
            self.cache.popitem(last=False)
```

This is concise and also runs in:

Time Complexity: $O(1)$ average per operation
Space Complexity: $O(\text{capacity})$

However, in interviews, you are often expected to implement the linked list manually to show you understand the underlying design.

2. Hash map plus timestamps

One might store a timestamp for each key and scan for the smallest timestamp during eviction.

However, eviction would take:

$O(n)$

So this does **not** satisfy the required $O(1)$ time complexity.

Key Pattern

This problem is a classic example of combining two data structures:

Hash Map + Doubly Linked List

Use this pattern when you need:

- Fast lookup by key
- Fast removal of arbitrary elements
- Fast tracking of order, especially recency or frequency